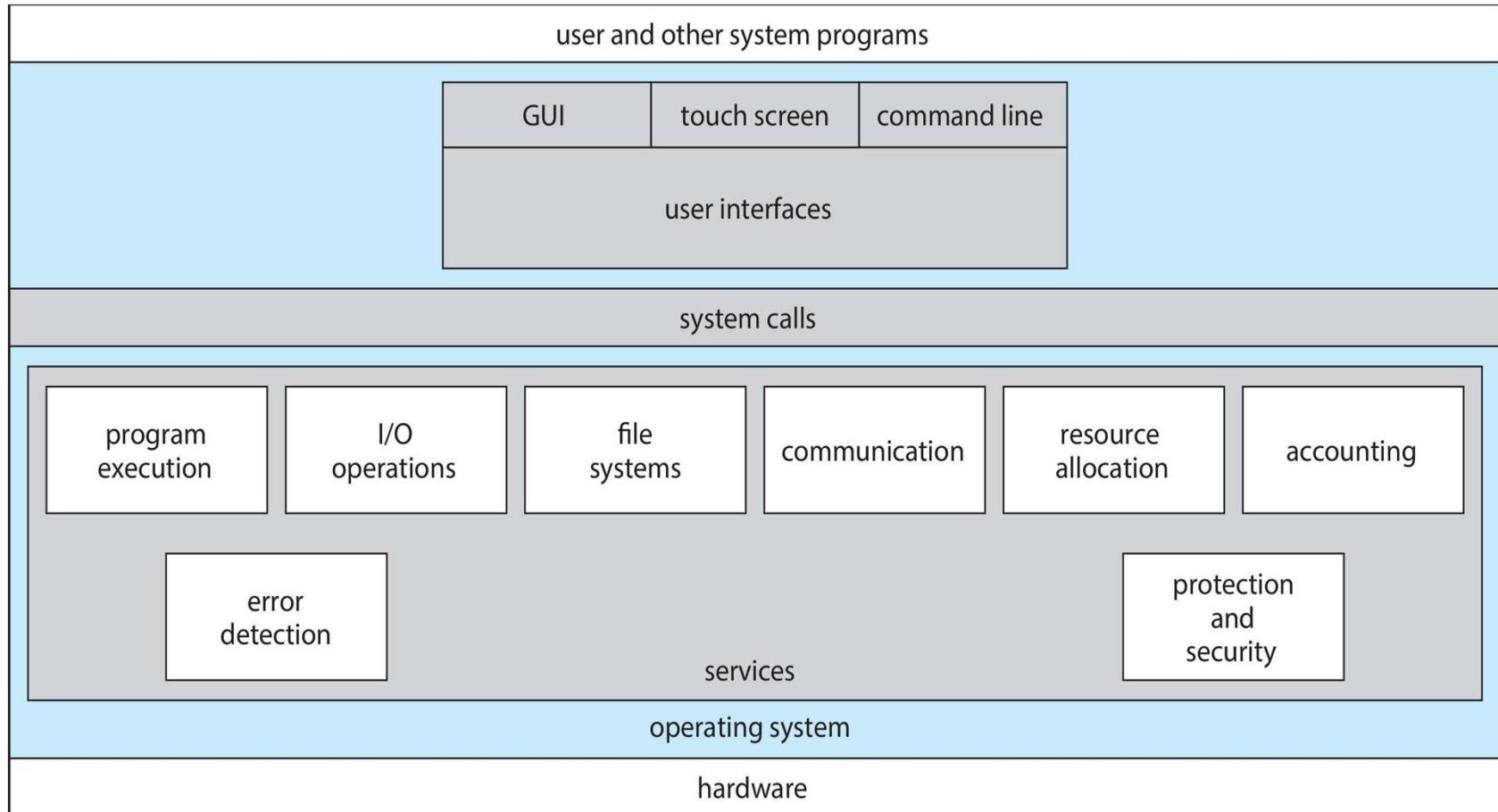


Chapter 2:

Operating System Services and Structures

A View of Operating System Services



Operating System Services

- Operating systems provide an environment for execution of programs and services to programs and users
- One set of operating system services provides functions that are helpful to the user:

User interface - Almost all operating systems have a user interface (UI).

- Command-Line (CLI), Graphics User Interface (GUI)

Program execution - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)

Operating System Services

I/O operations - A running program may require I/O, which may involve a file or an I/O device

File-system manipulation - Programs need to read and write files and directories, create and delete them, search them, list file information, permission management.

Communications – Processes may exchange information, on the same computer or between computers over a network

Error detection – OS needs to be constantly aware of possible errors

- May occur in the CPU and memory hardware, in I/O devices, in user program

Operating System Services

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing

Resource allocation – When multiple users or multiple jobs running concurrently, resources must be allocated to each of them

Accounting - To keep track of which users use how much and what kinds of computer resources

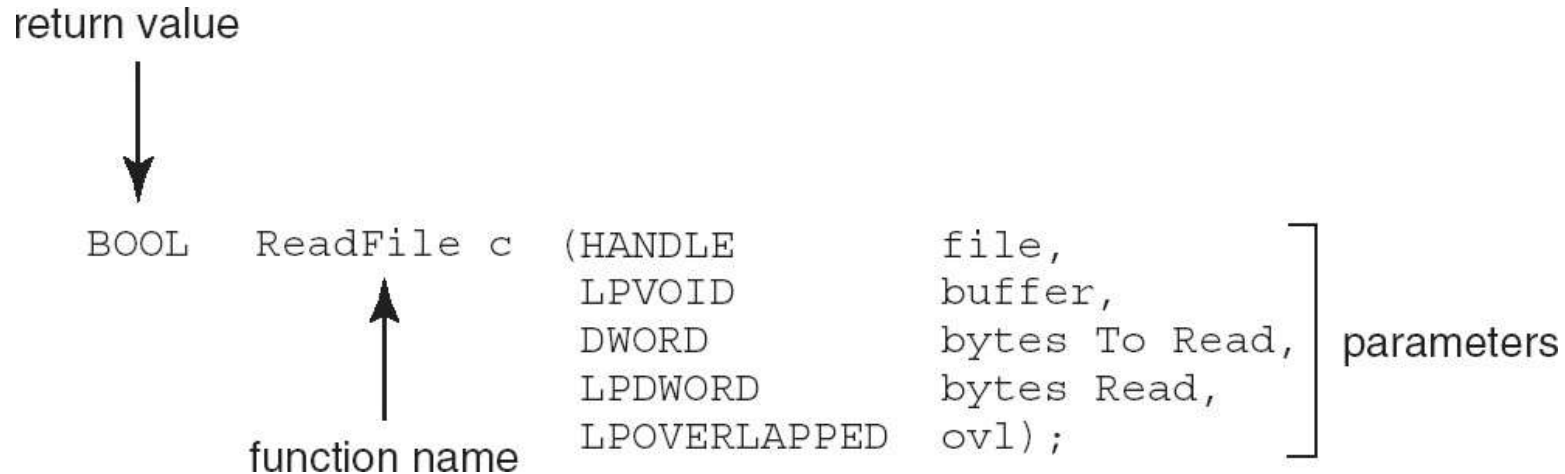
Protection and security - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other

System Calls

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use
- Three most common APIs are
 - Win32 API for Windows
 - POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X)
 - Java API for the Java virtual machine (JVM)

Example of System Calls

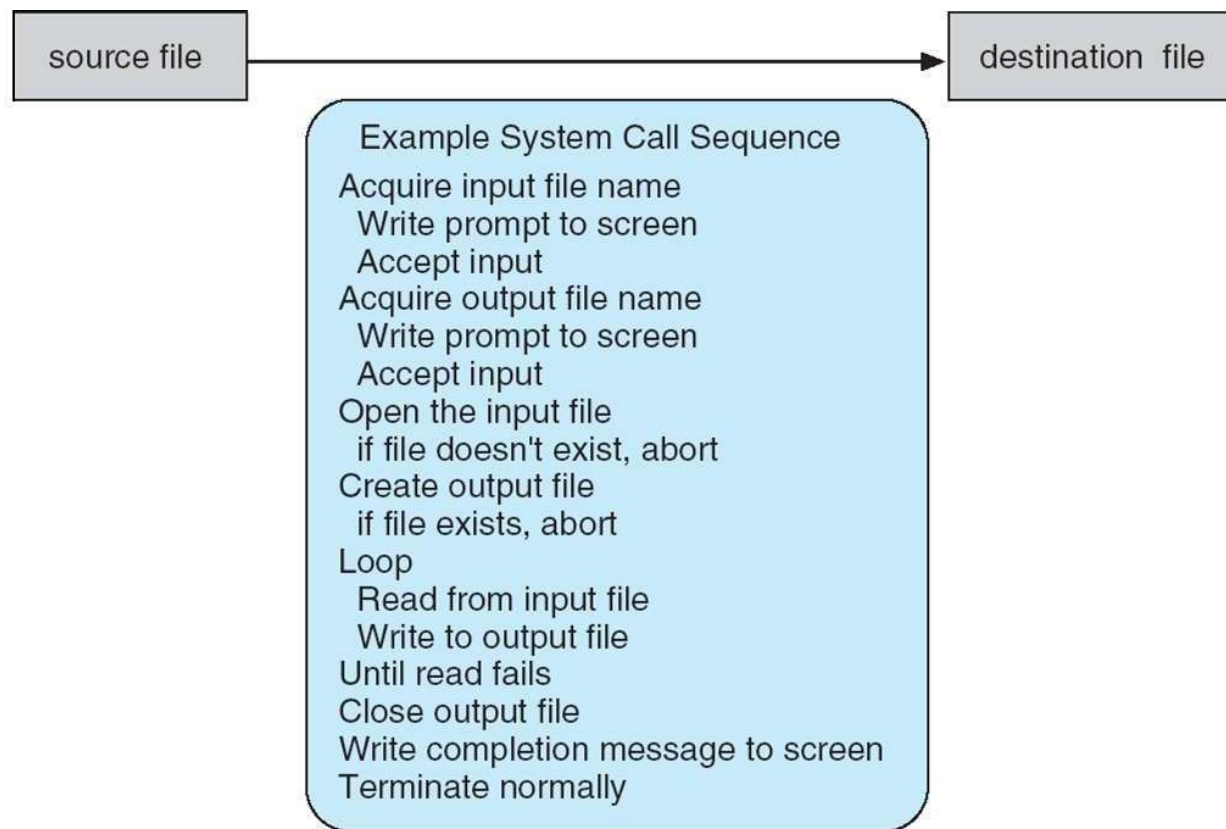
Consider the ReadFile() function in the Win32 API



Behind the scenes, the functions that make up an API typically invoke the actual system calls within the kernel on behalf of the application programmer.

Example of System Calls

- System call sequence to copy the contents of one file to another file



Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the man page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

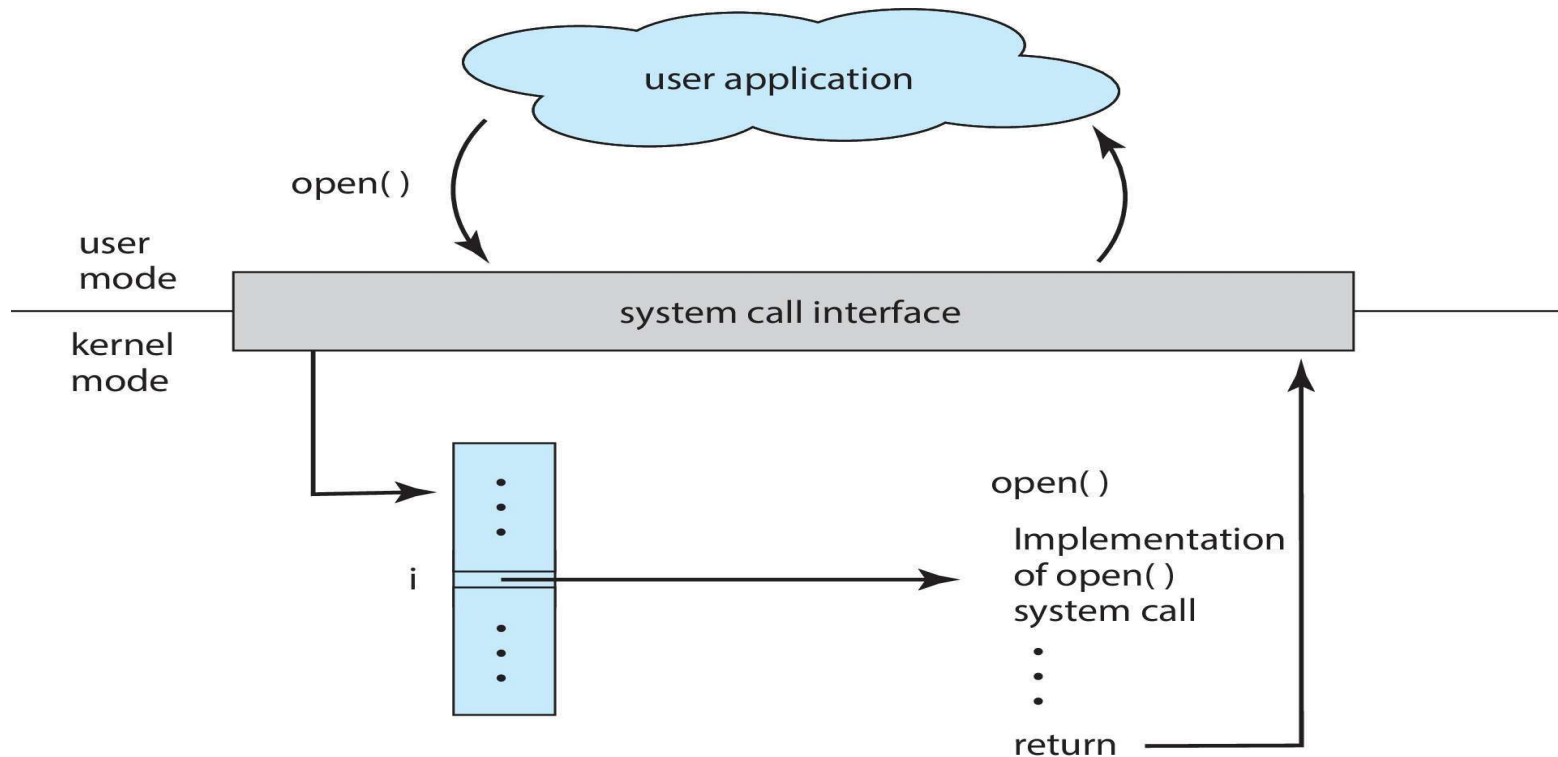
<pre>#include <unistd.h></pre>		
<pre>ssize_t</pre>	<pre>read</pre>	<pre>(int fd, void *buf, size_t count)</pre>
return value	function name	parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

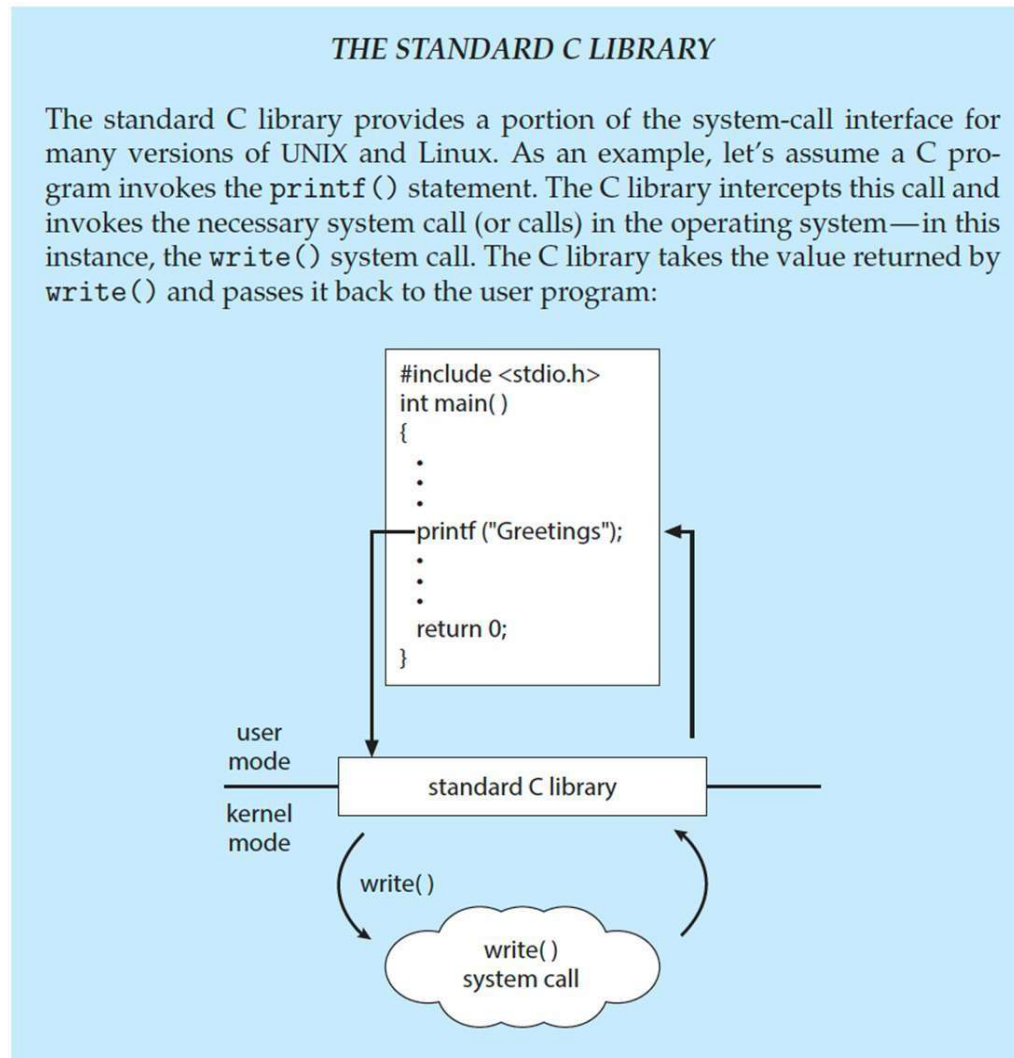
API – System Call – OS Relationship



System call interface: run-time support library (set of functions built into libraries included with compiler)

Standard C Library Example

- C program invoking `printf()` library call, which calls `write()` system call



Types of System Calls

□ Process control

- end, abort
- load, execute
- create process, terminate process
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory

□ File management

- create file, delete file
- open, close file
- read, write, reposition
- get and set file attributes

□ Device management

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

□ Information maintenance

- get time or date, set time or date
- get system data, set system data
- get and set process, file, or device attributes

□ Communications

create, delete communication connection
send, receive messages
transfer status information
attach and detach remote devices

Examples of Windows and Unix System Calls

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

System Services

- Provide a convenient environment for program development and execution.

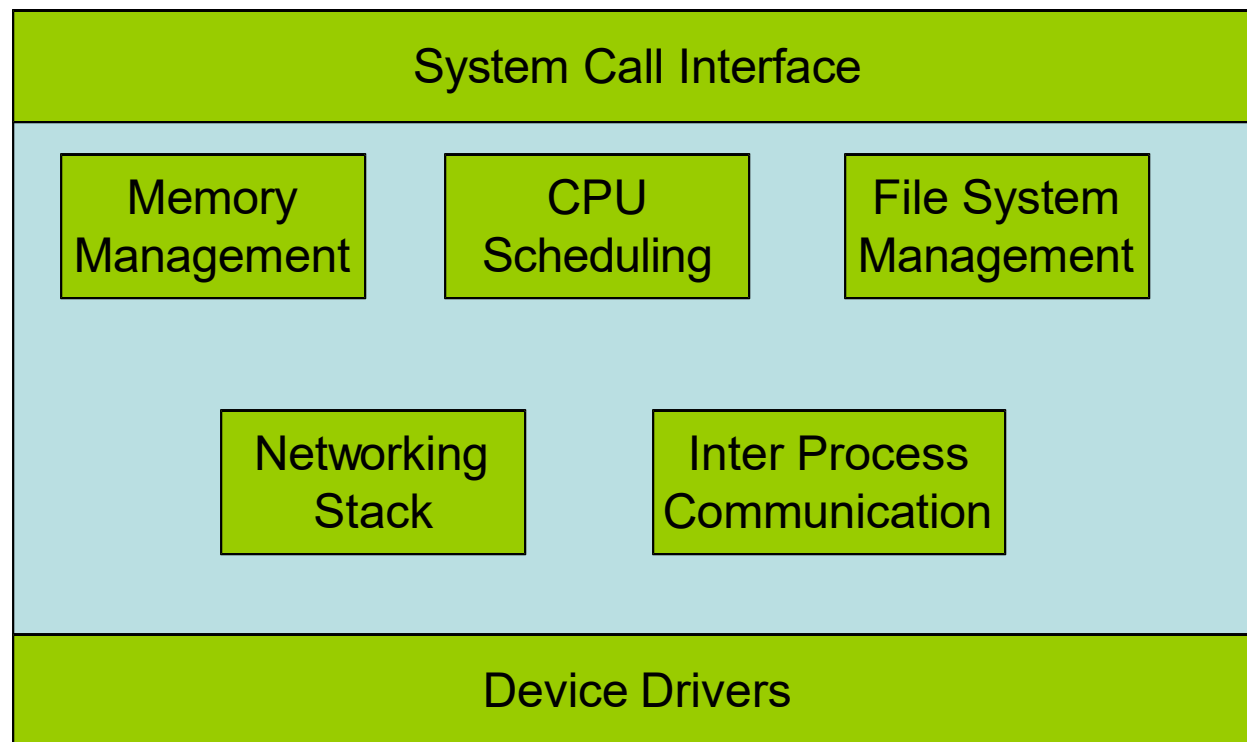
They can be divided into:

- **File management**
 - **Status information**
 - **File modification**
 - **Programming-language support**
 - **Program loading and execution**
 - **Communications**
 - **Background Services**
 - **Application programs**
-
- Most users' view of the operation system is defined by system programs, not the actual system calls

Implementation

- Much variation
 - Early OSes in assembly language
 - Then system programming languages like Algol, PL/1
 - Now C, C++
- Actually usually a mix of languages
 - Lowest levels in assembly
 - Main body in C
 - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts

What goes into an OS?



The Architecture Question

We have many OS components. How should we organize them inside the operating system?

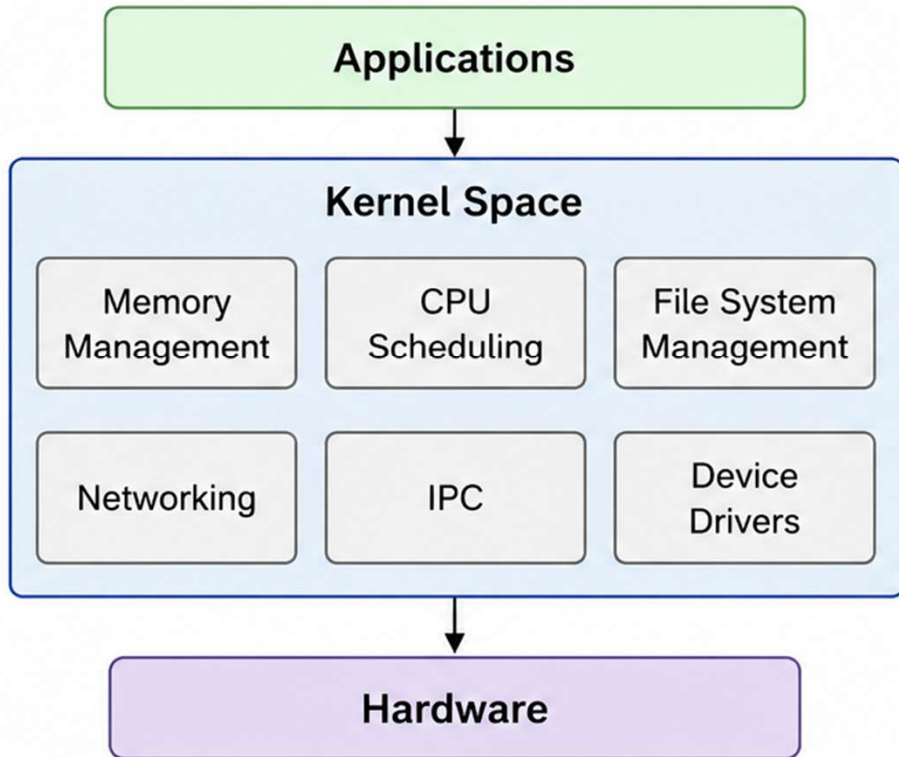
-  Put everything inside the kernel?
-  Divide the OS into layers?
-  Keep only essential functions in the kernel?
-  Build the kernel from independent modules?



Different answers → Different OS architectures

Monolithic Architecture

All core OS components run in kernel space.



Advantages

- Components communicate via direct procedure calls
- Efficient (low communication overhead)
- High performance



Disadvantages

- Large kernel
- Harder to maintain and verify
- A bug in any component can crash the entire system

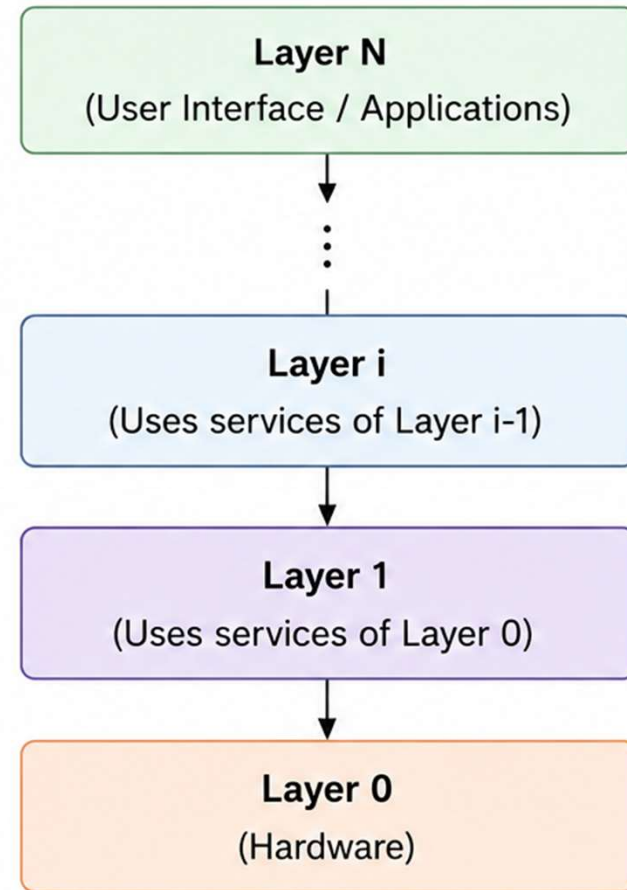
Examples: Linux (traditional), MS-DOS, xv6, traditional UNIX

Layered Architecture

Instead of letting every component interact freely, **organize the OS into layers**.

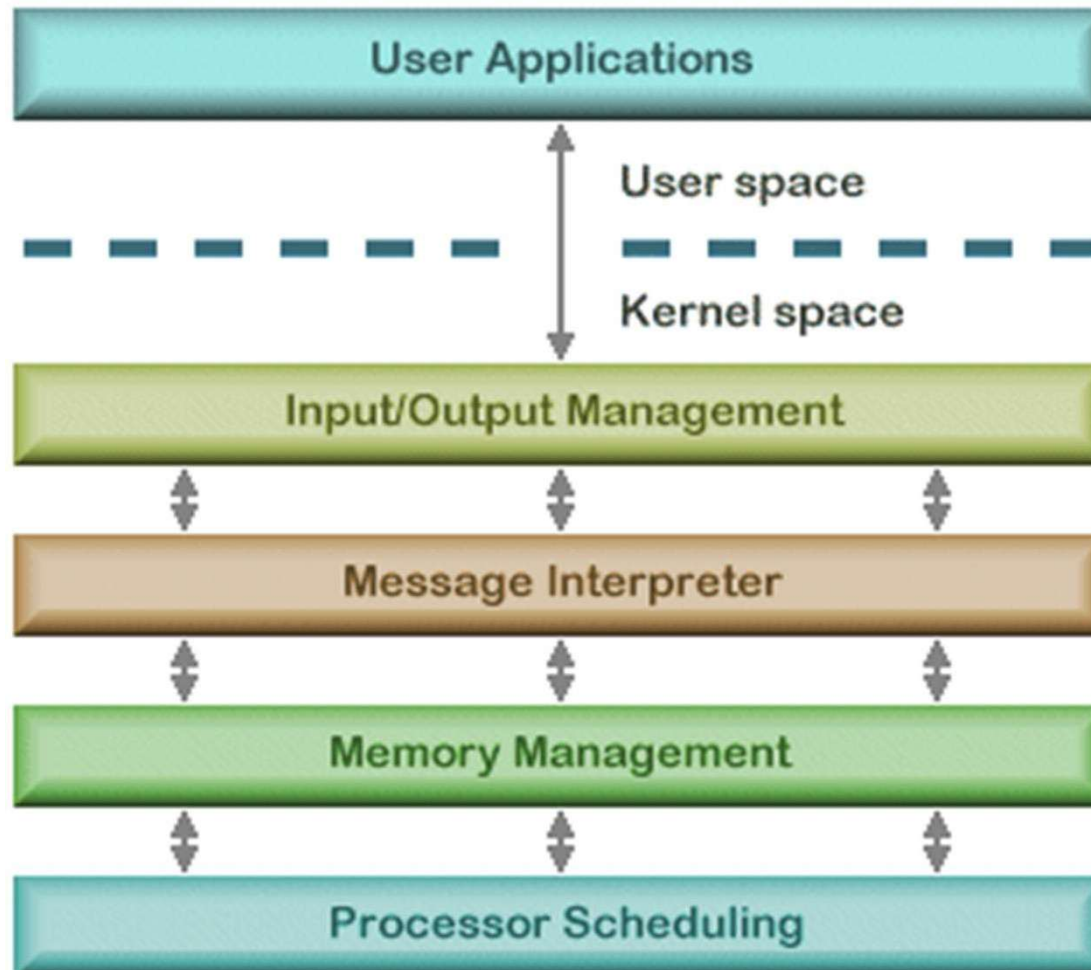
Definition

- Each layer is **built on lower layers** and **uses the functions/services** provided by the lower-level layers.



Key Idea: A layer provides services to the layer above it and relies on the services of the layer below it.

Layered Approach



Layered: Trade-offs



Advantages

-  Modularity
-  Easier construction
-  Easier debugging

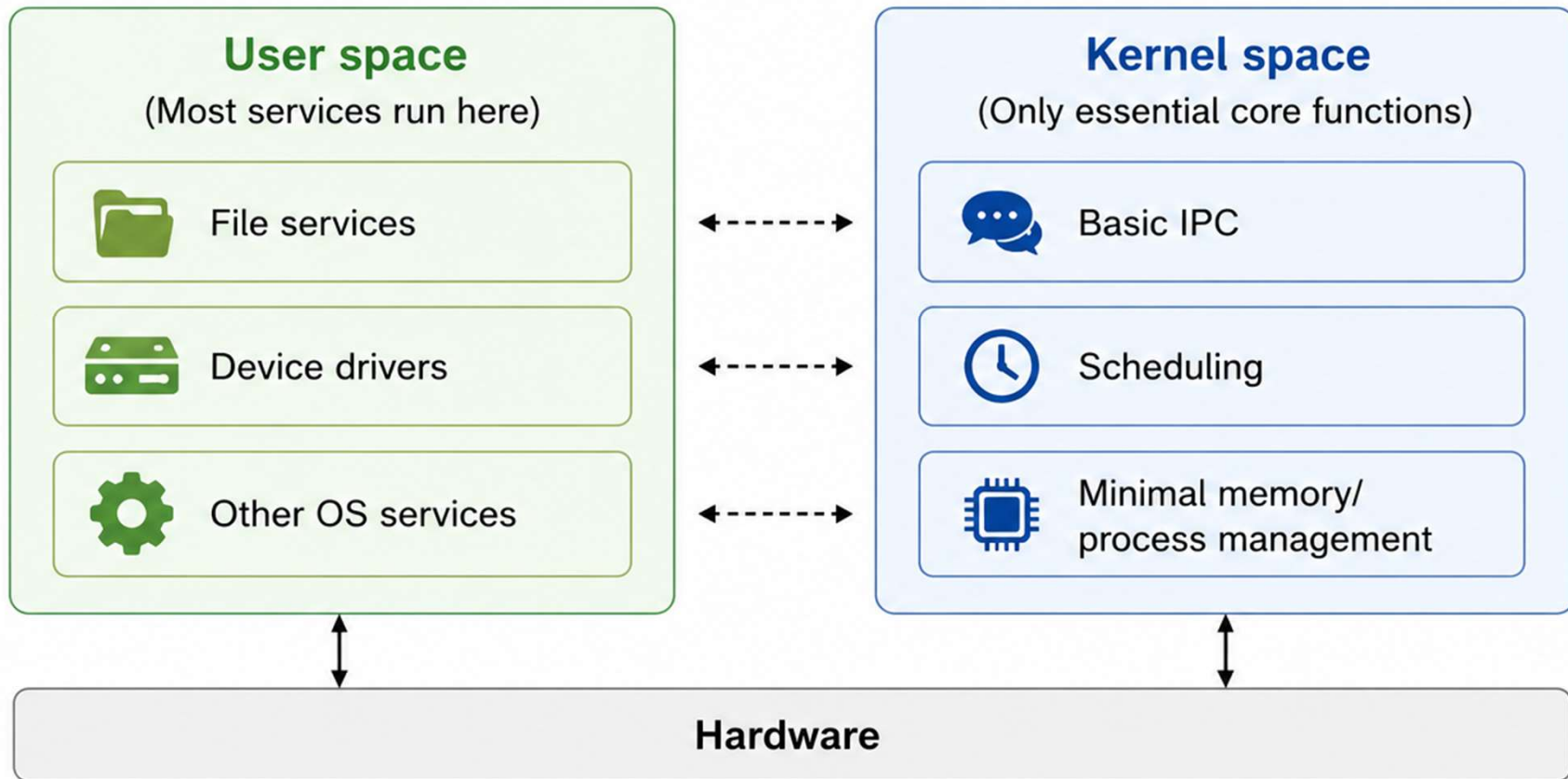


Disadvantages

-  Difficult to define the correct layers
-  Additional overhead
-  Less flexible

Microkernel: A Different Idea

What if we move as much as possible **OUT** of the kernel?

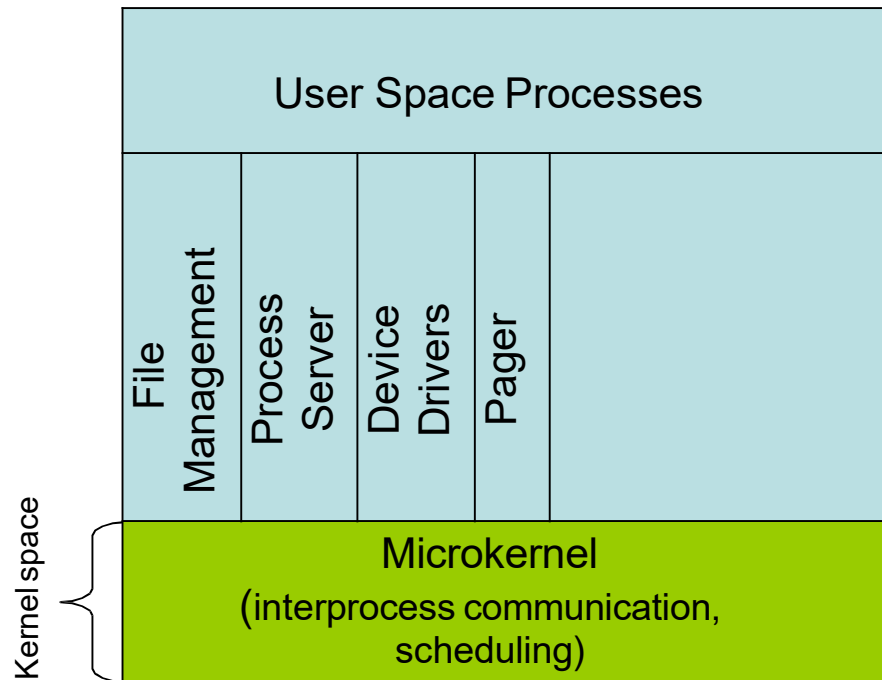


Only **essential core functions** remain in the kernel; less-essential services execute in **user mode**.

Microkernels

- Only absolutely essential core OS functions should be in the kernel.
- Less essential services and applications are built on the microkernel and execute in user mode.
- Example: [Mach](#) → Carnegie Mellon University
- Typically, microkernels provide minimal process and memory management, in addition to a [communication facility](#) between the client program and the various services that are also running in user space

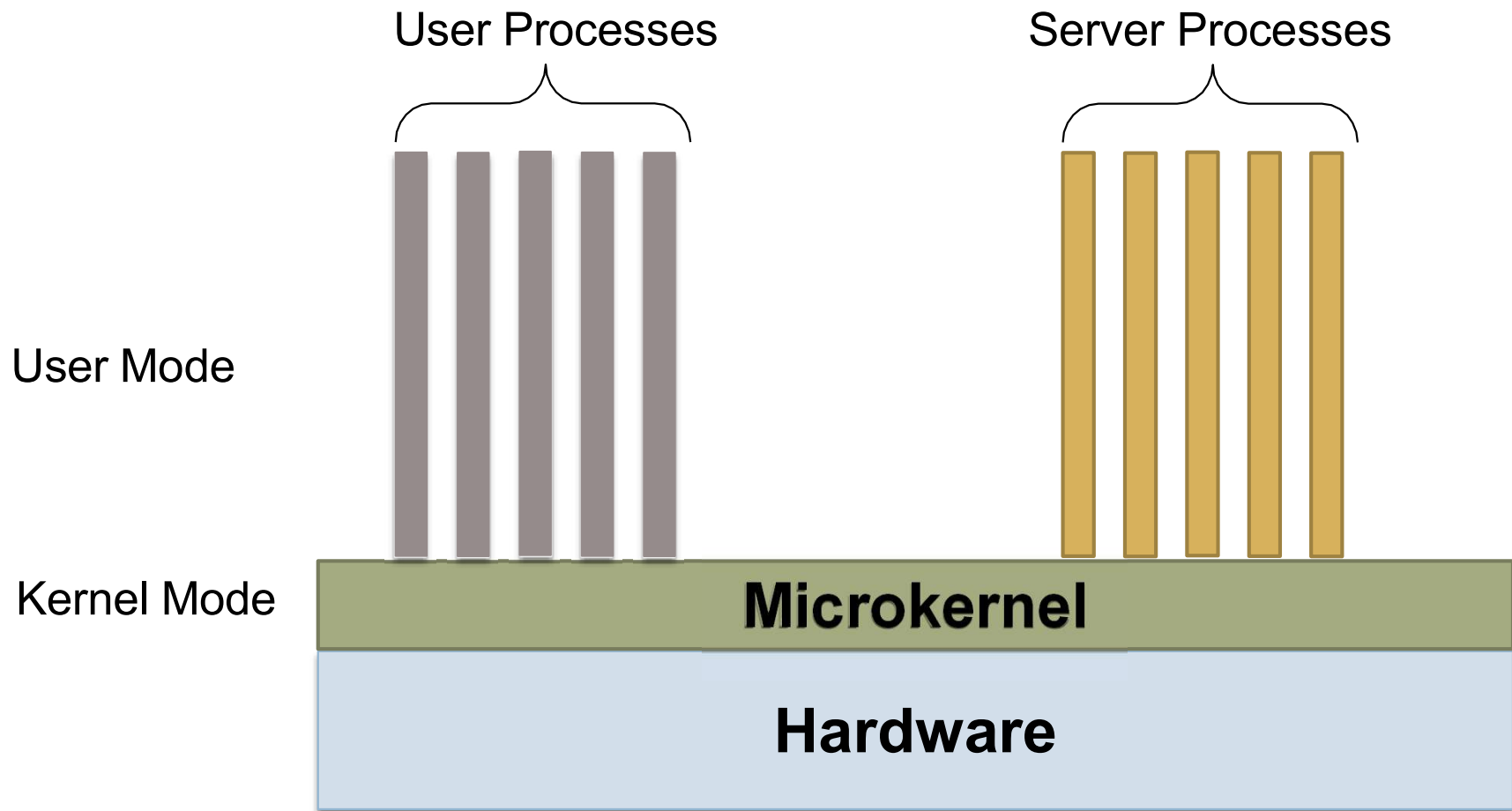
Microkernel



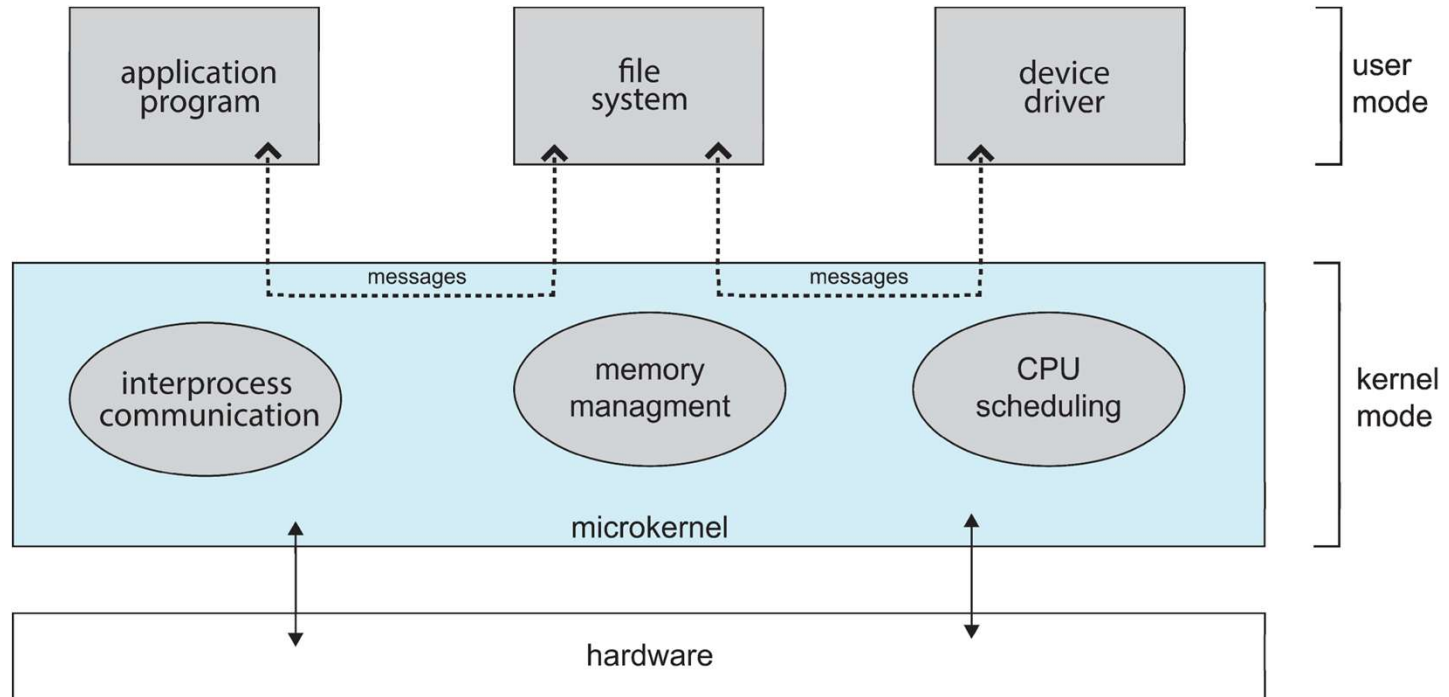
Eg. QNX and L4

- Highly modular.
 - Every component has its own space.
 - Interactions between components strictly through well defined interfaces (no backdoors)
- Kernel has basic inter process communication and scheduling
 - Everything else in user space.
 - Ideally kernel is so small that it fits the first level cache

Microkernels



Microkernel System Structure

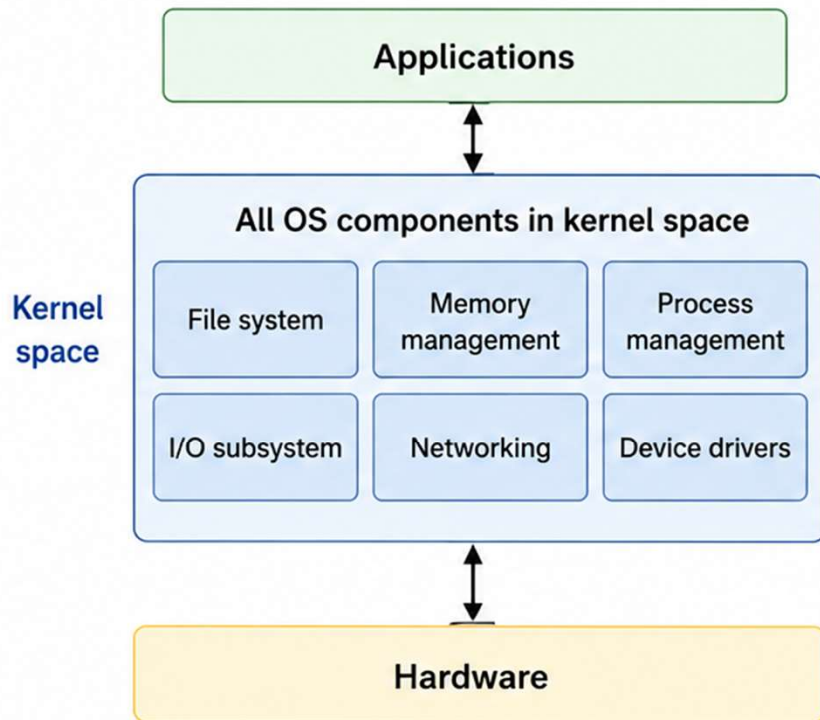


Microkernels

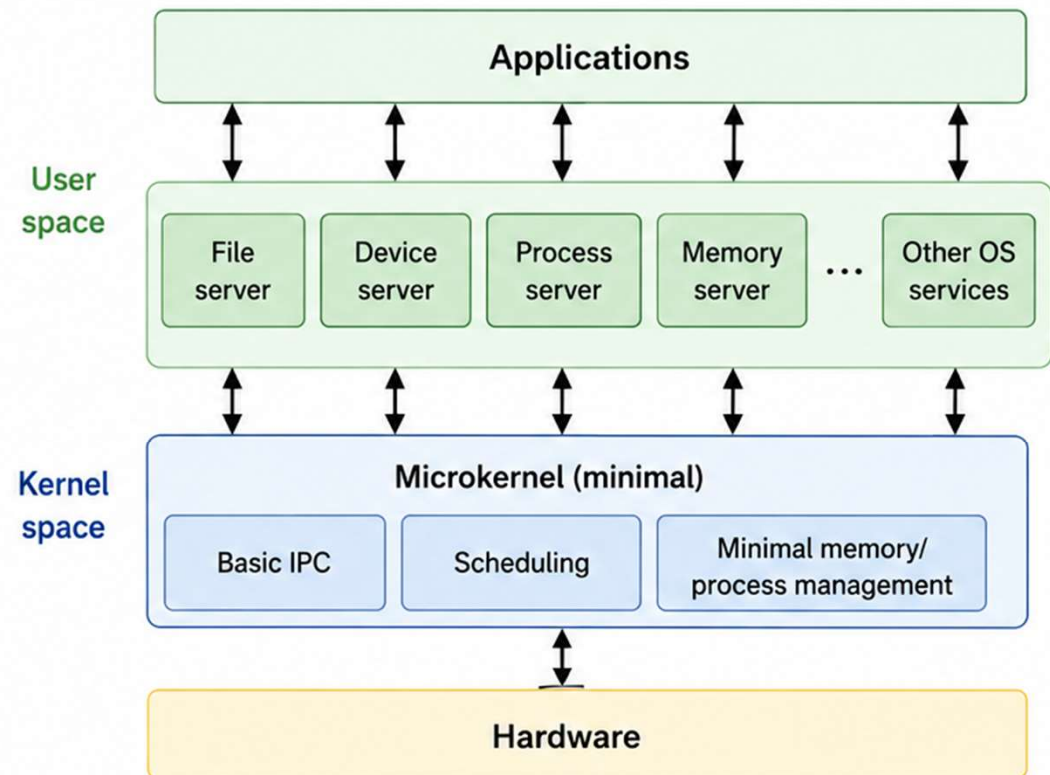
- Moves as much from the kernel into user space
- **Mach** is an example of **microkernel**
 - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**
- **Benefits:**
 - Easier to extend a microkernel
 - Easier to port the operating system to new architectures
 - More reliable (less code is running in kernel mode)
 - More secure
- **Detriments:**
 - Performance overhead of user space to kernel space communication

Monolithic vs. Microkernel

Monolithic Architecture



Microkernel Architecture



Think about it:

- Which kernel is smaller?
- Which architecture requires more communication across the user/kernel boundary?

Microkernel vs. Layered Approach

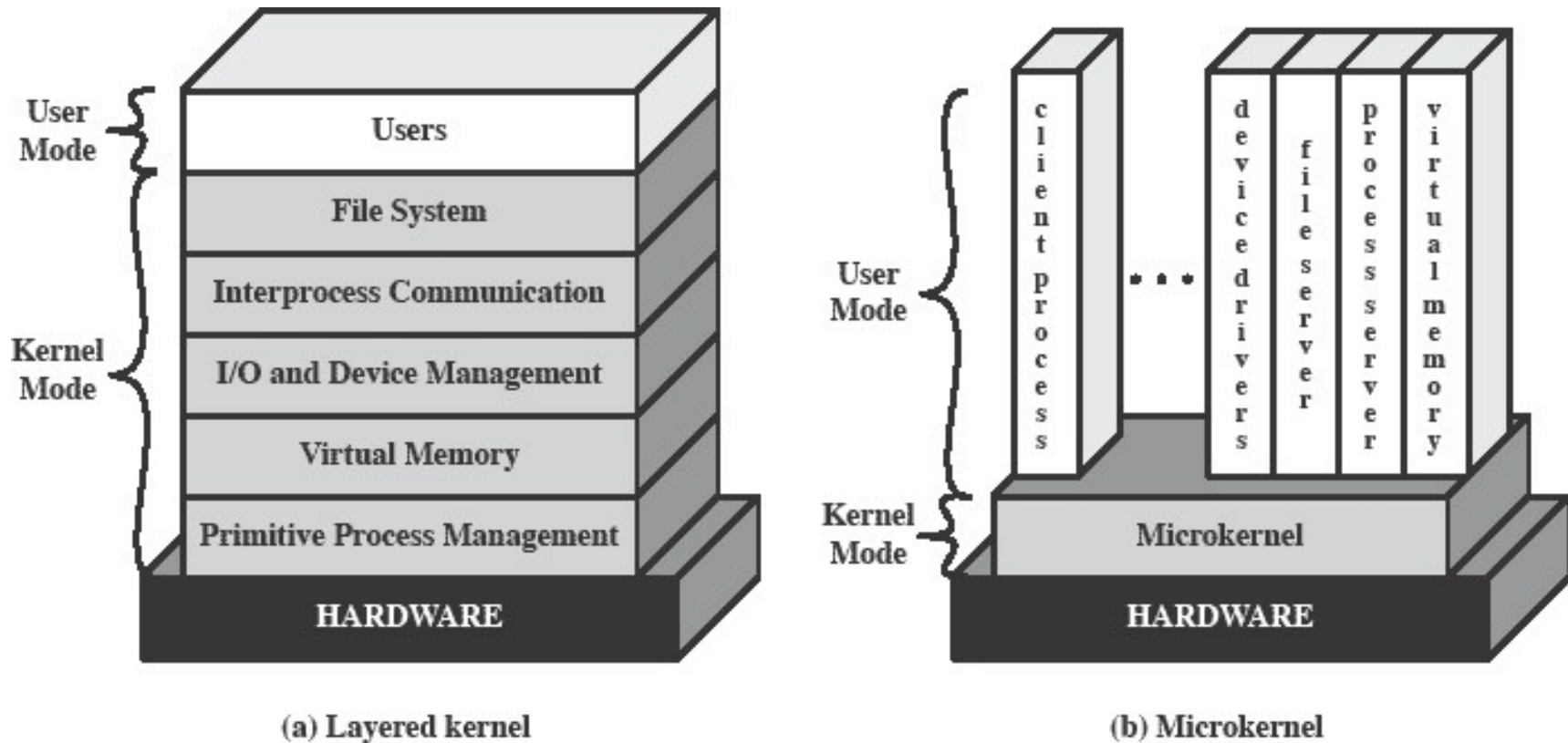


Figure 4.10 Kernel Architecture

Architecture Comparison

Aspect	Monolithic	Layered	Microkernel
 Inter-process communication	Direct procedure calls within kernel	Through layers (layer-to-layer calls)	Message passing (user ↔ kernel)
 Memory management	In kernel space	Usually in a lower layer (or kernel layer)	Minimal in kernel; most in user space
 Stability (Reliability)	Lower – a bug in kernel can crash the system	Better – faults limited to the layer	Higher – failures in servers don't crash the kernel
 I/O Communication (Interrupts)	Handled in kernel	Handled in lower layers (up to kernel)	Handled by user-space servers; via IPC
 Extendibility	Harder (change requires kernel modification)	Moderate (add/change within a layer)	Easier (add new servers in user space)
 Speed (Performance)	Faster (direct calls, no extra overhead)	Moderate (some layer overhead)	Slower (IPC introduces overhead)



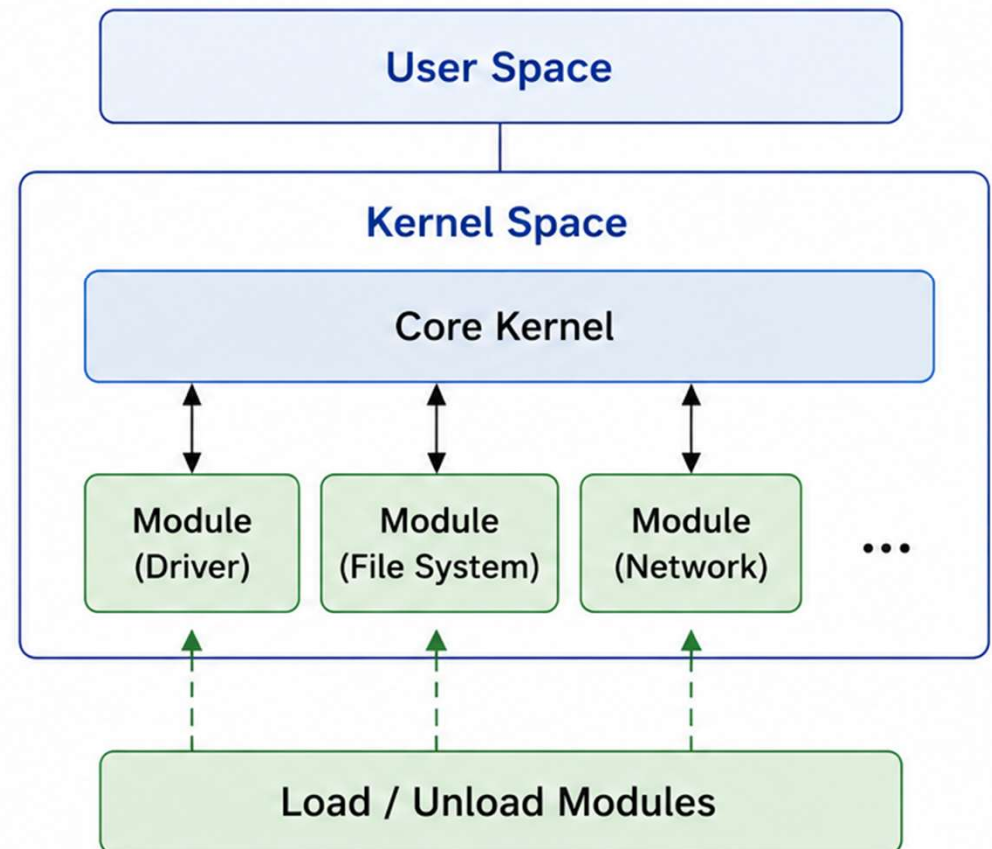
Key Takeaway: Each architecture makes a different trade-off between performance, reliability, complexity, and flexibility.

Loadable Kernel Modules

Instead of a fixed monolithic kernel, add or remove components as needed at **run time**.

Key Ideas

- **Core kernel** stays in memory.
- Modules can be **loaded** when needed and **unloaded** when not needed.
- Provides **flexibility** without rebooting the system.
- **Examples**: device drivers, file systems, network protocols.



Hybrid Systems



Modern operating systems rarely follow one architecture perfectly.

They combine ideas from multiple approaches to achieve **performance**, **security**, and **usability**.



Linux

(Monolithic + Modular)

- Monolithic kernel for performance
- Loadable Kernel Modules (LKMs) for flexibility
- Wide range of drivers, file systems, and network modules
- Used in servers, desktops, and embedded systems



Solaris

(Monolithic + Modular)

- Monolithic kernel for high performance
- Loadable kernel modules for device drivers and file systems
- Advanced features (ZFS, zones, scheduling)
- Used in enterprise and server environments



Windows

(Hybrid Characteristics)

- Hybrid kernel design (microkernel-like components inside the kernel)
- Executive, device drivers, and subsystems in kernel mode
- User-mode services communicate via efficient IPC mechanisms
- Focus on compatibility and performance



macOS

(Mach + BSD + Other Components)

- Built on the Mach microkernel
- Adds BSD UNIX subsystems and I/O Kit (drivers)
- Combines performance, security, and rich user experience
- Used on Apple hardware platforms

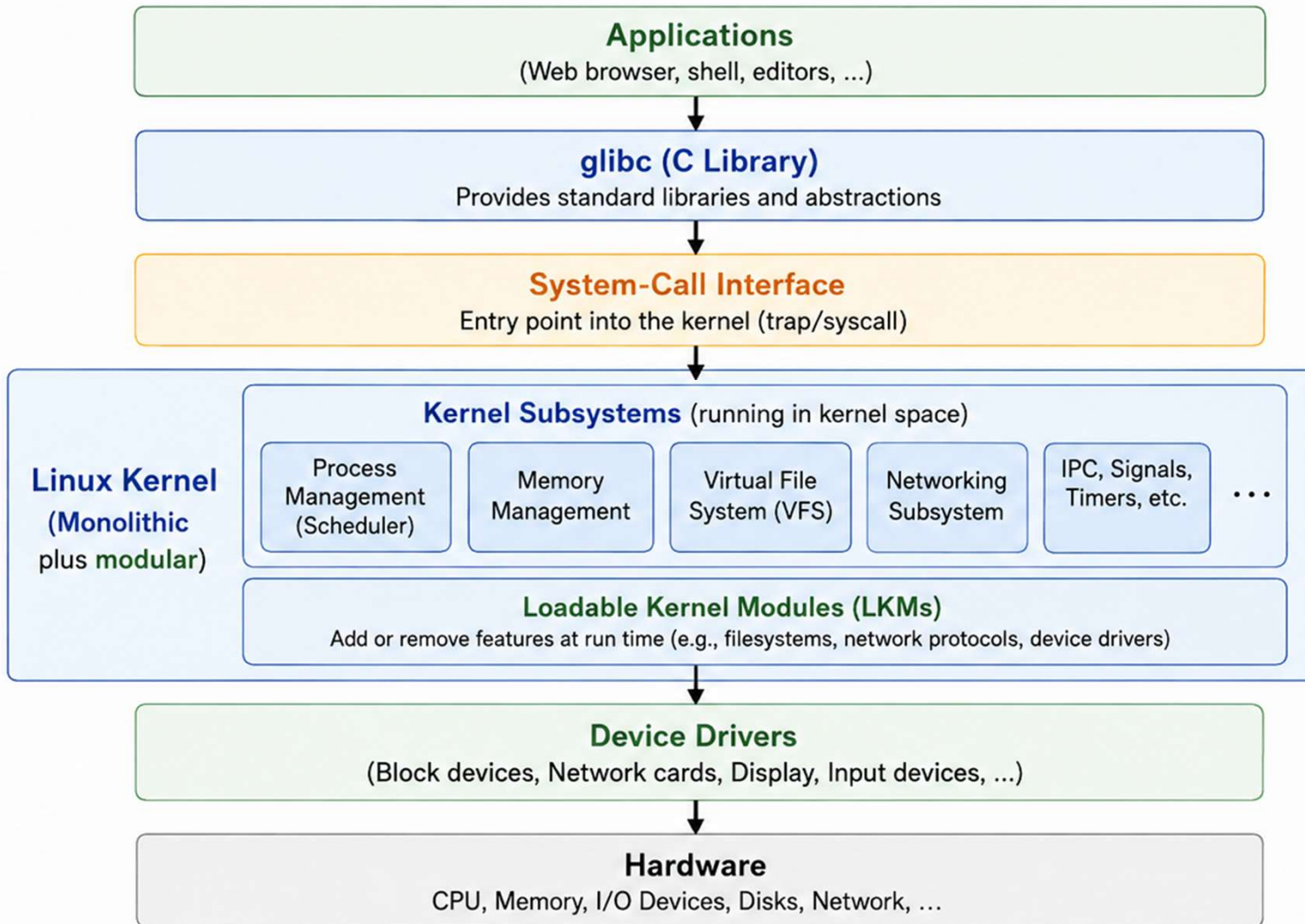


Key Takeaway: Real operating systems are hybrids.

They choose the right mix of ideas to balance **performance**, **security**, **flexibility**, and **usability**.

Linux: Putting It Together

Linux uses a **Monolithic** plus **Modular** design.



Final Comparison

Aspect	 Monolithic	 Layered	 Microkernel	 Modular
 Kernel size	Large	Depends	Small	Large/core + modules
 Organization	Components together	Layers	Minimal kernel + servers	Kernel modules
 Communication	Direct	Layer-to-layer	Message passing	Defined interfaces
 Performance	High	Some overhead	More IPC overhead	High
 Maintainability	Harder	Easier	Easier	Flexible
 Main idea	Everything inside	Organize vertically	Move services outside	Plug components in